

Summary of book Privacy-Preserving Machine Learning (Chang, Zhuang and Samaraweera, Manning 2023).

Implementation of code snippets.

Code implementations can be found in the notebooks folder on the link:
github.com/popchanovska/differential-privacy-intro.

Topics covered in code implementation

- Task 1 — Differential privacy mechanisms: binary (randomized response), Laplace, and exponential mechanisms.
- Task 2 — Differentially private machine learning: naive Bayes, logistic regression, linear regression, k-means clustering.
- Task 3.1 — Local differential privacy: randomized response for binary survey answers.
- Task 3.2 — Local differential privacy: direct encoding for categorical survey answers.
- Task 3.3 — Local differential privacy: histogram encoding (thresholding, THE) for numerical/continuous data.
- Task 4.1 — Data sanitization: generalization, suppression, perturbation, and anatomization.
- Task 4.2 — Anonymity models: k-anonymity, l-diversity, and t-closeness on one recursive-partitioning engine.
- Task NEW — A practical PyDP (OpenMined) comparison: true vs. differentially private answers to ten simple questions.

Datasets used for code snippets

All datasets are pulled automatically from Kaggle via the kagglehub library. The datasets used across the notebooks are:

Notebook	Kaggle dataset
Task 1 — DP mechanisms	brendan45774/test-file (Titanic)
Task 2 — DP ML methods	miadul/lifestyle-and-health-risk-prediction
Task 3.1 — Randomized response	laveshjadon/ai-impact-on-students
Task 3.2 — Direct encoding LDP	laveshjadon/ai-impact-on-students
Task 3.3 — Histogram encoding LDP	laveshjadon/ai-impact-on-students
Task 4.1 — Data sanitization	deepeshkansotia/hantavirus-transmission-and-risk-dataset
Task 4.2 — Anonymity models	deepeshkansotia/hantavirus-transmission-and-risk-dataset
Task NEW — PyDP comparison	laveshjadon/ai-impact-on-students

Chapter 1 Summary — Privacy-preserving machine learning

Subsection 1.1 - Privacy complications in the AI era

- This subsection shows several real examples where personal data was leaked or misused.
- In the Facebook-Cambridge Analytica case, a quiz game on Facebook collected data from the people who played it and it also took data from their friends.
- Google developed a tool called RAPPOR, which helps collect user answers while keeping each person's data private.
- In 2021, Google launched its Privacy Sandbox initiative in the Chrome browser, replacing third-party cookies and setting clear limits on how advertising companies can use private data.
- Apple says it first introduced its own well-developed privacy techniques with iOS 11, using them to study things like emoji preferences and usage among iPhone users.

Subsection 1.2 - The threat of learning beyond the intended purpose

- A large amount of our data is collected and stored online every day. This includes web search queries, browsing history, online purchase records, movie preferences and location check-ins. This data is also exposed to insider attacks, because workers at data-mining companies can access it, which is a very serious problem.
- Extra information can sometimes be extracted from private data even when it has been anonymized, or even when the datasets and ML models are hidden and only the test results are shared. At the same time, machine learning is data driven, so we need training samples to build a model.
- The goal is to use private data to train a model while still preventing the algorithm from learning anything sensitive.
- Privacy must be implemented in a systematic way rather than through random methods, because many additional threats have to be considered.
- Organizations follow privacy standards, such as HIPAA, PCI DSS, FERPA and the European Union's GDPR.

Subsection 1.3 - Threats and attacks for ML systems

- It is still difficult to raise public awareness of these privacy technologies, mainly because there are few well-organized tutorials and books that explain the concepts clearly.
- It is important to store private data on the server in encrypted form and we should never send raw data to the server in its original form.
- One important example is when Al-Rubaie and his team studied reconstruction attacks that used raw gesture data from users' authentication profiles in mobile-based continuous authentication systems.
- Simple statistics like the count, mean and median age of a population, broken down by a few demographics, can allow someone with a personal computer to accurately rebuild the personal data of almost the entire survey population.
- With white-box access, malicious insiders have direct access to the ML model, so even with limited knowledge they can still infer what is happening inside.
- With black-box access, the adversary cannot reach the model directly, but they can listen to the incoming requests and the responses the model produces for a given input.

Subsection 1.4 - Securing privacy while learning from data

- Many privacy-enhancing techniques focus on letting several parties train ML models together without releasing their private data in its original form.
- This collaboration can be done using cryptographic approaches or by releasing differentially private data through perturbation techniques.
- Some techniques include differential privacy, local differential privacy (LDP), privacy-preserving synthetic data generation, privacy-preserving data mining techniques and compressive privacy.

Chapter 1 coding task — Differential privacy mechanisms (Task 1)

Dataset used: *brendan45774/test-file (Titanic passenger dataset, 418 rows)*

This notebook builds the three foundational mechanisms from the book by hand and applies each to the column type it is meant for: the binary mechanism on the binary Survived column, the Laplace mechanism on numerical columns (a count and a mean), and the exponential mechanism on the categorical Embarked column. All three are then compared across several privacy budgets (epsilon) to show the accuracy/privacy trade-off.

1. Binary mechanism (randomized response) — Survived

The binary mechanism answers a yes/no question truthfully with a probability p (a biased coin), and randomly otherwise, which gives every respondent plausible deniability.

```
def randomized_response_mechanism(x, p):
    if random() < p:      # first coin flip
        return x
    elif random() < p:    # second coin flip
        return 1
    else:
        return 0
```

What this code does: Defines the randomized-response mechanism: with probability p it returns the true 0/1 answer, otherwise it returns a second coin flip as the reported answer.

The mechanism was run on the 418 Survived values for $p = 0.5 \dots 0.9$ (equivalently $\epsilon = \ln(1 + p/((1-p)^2)) \approx 1.10 \dots 4.51$):

p	epsilon	Reported 1s	True 1s
0.5	1.099	189	152
0.6	1.558	185	152
0.7	2.172	191	152
0.8	3.045	188	152
0.9	4.511	182	152

2. Laplace mechanism — survivor count and mean Fare

The Laplace mechanism perturbs a numeric query result $f(\text{data})$ by adding noise drawn from a Laplace distribution scaled by sensitivity/epsilon. It was applied to two queries: the count of survivors (sensitivity 1) and the mean Fare (a bounded-range mean, sensitivity = range/n).

```
def laplace_mechanism(data, f, sensitivity, epsilon):
    return f(data) + laplace(0, sensitivity / epsilon)
```

What this code does: Applies the queried function f to the data and adds zero-mean Laplace noise scaled by sensitivity/epsilon.

Each configuration was averaged over 200 releases (true survivor count = 152, true mean Fare = 35.627):

epsilon	Count: mean	Count: std	Mean Fare: mean	Mean Fare: std
0.01	148.43	156.20	44.332	162.7006
0.1	151.00	14.37	36.390	14.4079

epsilon	Count: mean	Count: std	Mean Fare: mean	Mean Fare: std
0.5	151.90	3.02	35.563	3.0018
1.0	152.02	1.32	35.597	1.8506
5.0	152.02	0.28	35.594	0.4179

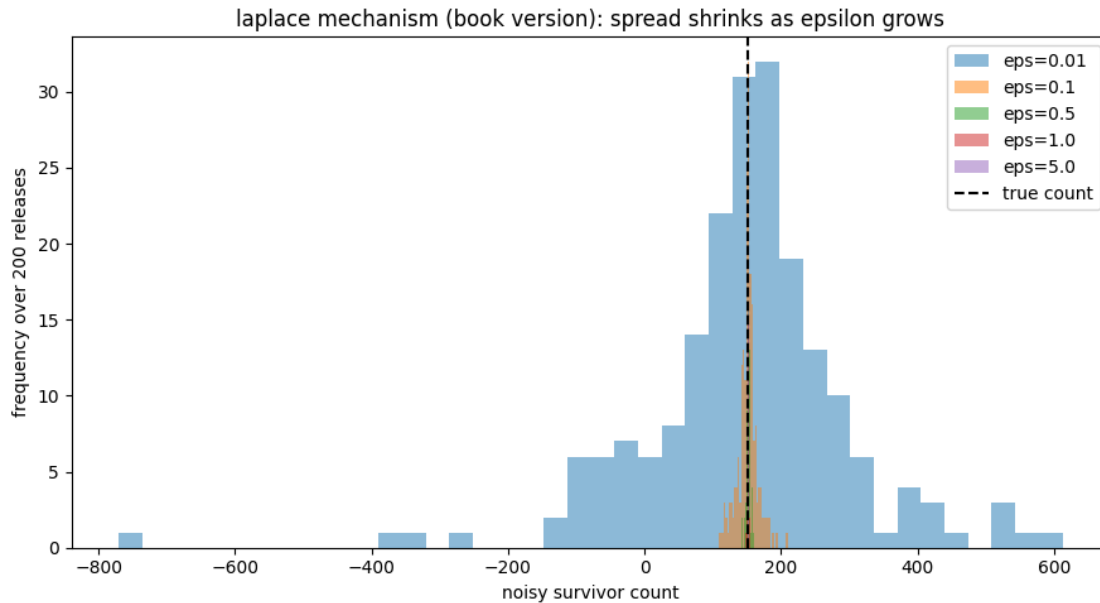


Figure 1.1 — Distribution of the noisy survivor count over 200 releases for five privacy budgets; the spread visibly shrinks as epsilon grows, converging on the true count (dashed line).

3. Exponential mechanism — most common Embarked port

The exponential mechanism selects a categorical output with probability proportional to $\exp(\text{epsilon} \cdot \text{utility} / (2 \cdot \text{sensitivity}))$, where utility here is each port's passenger count.

```
def most_common_embarked_exponential(x, A, H, sensitivity, epsilon):
    utilities = H(x, A)
    probabilities = [np.exp(epsilon * u / (2 * sensitivity)) for u in utilities]
    probabilities = probabilities / np.linalg.norm(probabilities, ord=1)
    return np.random.choice(A, 1, p=probabilities)[0]
```

What this code does: Computes a sampling probability for each port proportional to $\exp(\text{epsilon} \times \text{utility})$, then draws one port from that distribution.

True counts: S = 270, C = 102, Q = 46 (true mode = S). Two utility scalings were tested over 5000 draws each — the book's utility/100 scaling (reused from a much larger census example) and an unscaled utility equal to the raw count:

epsilon	P(true mode) — utility/100	P(true mode) — unscaled utility
0.01	0.335	0.572
0.1	0.350	1.000
0.5	0.442	1.000
1.0	0.558	1.000

epsilon	P(true mode) — utility/100	P(true mode) — unscaled utility
5.0	0.981	1.000

Comments and insights

How the privacy budget impacts utility

- **Binary mechanism:** at $p = 0.5$ (epsilon = $\ln 3$) a quarter of answers are flipped, pulling the reported count toward $n/2$; as p rises toward 0.9 the count approaches the true value. The noise is unbiased in aggregate but every individual answer stays deniable.
- **Laplace mechanism:** the released value $f(\text{data}) + \text{Lap}(\text{sensitivity}/\text{epsilon})$ is unbiased at every budget, but its standard deviation is $\text{sensitivity}/\text{epsilon}$ and explodes as epsilon shrinks. The survivor count (sensitivity 1) is swamped at epsilon = 0.01 but near-exact at epsilon = 5.0; the bounded mean-Fare query has much smaller sensitivity, so it tolerates small budgets far better.
- **Exponential mechanism:** with the book's counts/100 scaling the utilities are tiny on this small dataset, so the mode barely wins even at epsilon = 5 — an artifact of reusing a census-scale constant on a small file. Dropping the /100 restores the expected behaviour: near-uniform selection at tiny epsilon (strong privacy, useless answer), concentrating on the true mode as epsilon grows.
- **Overall:** small epsilon buys privacy at the cost of utility (noisier counts, wider spreads, near-random selection); large epsilon recovers utility while weakening the guarantee — and sensitivity scales the effect, so low-sensitivity queries stay usable at much smaller budgets.

Different types of privacy attacks

1. Reconstruction attacks

Here the attacker gains an advantage by having outside knowledge of the feature vectors or the data used to build the ML model. Using this knowledge, they try to rebuild the original data behind the model. These attacks usually require direct access to the model, which is known as white-box access.

2. Model inversion attacks

Some ML models, such as neural networks, do not store feature vectors inside them, so the attacker's knowledge is more limited. Even so, the attacker can listen to the model's responses for new test samples. They then use these responses to recreate data that resembles the original feature vectors used to train the model.

3. Membership inference attacks

This is an extended version of the model inversion attack. Using the model's output, the attacker tries to find out whether a specific sample was part of the training dataset. For example, the attacker might use someone's record to learn whether their data was used to build the model, which can reveal very sensitive information about them.

4. De-anonymization (re-identification) attacks

Even when a dataset has been anonymized, it can still be unsafe. The attacker uses background knowledge to match records back to real people and discover their identities.

5. Correlation attacks

Here the attacker links several different data sources together. This produces more private information about each user than any single dataset held on its own. For example, by combining sources, an attacker could learn which patient bought medication from which pharmacy.

6. Identification attacks

While a correlation attack gathers more private information in general, an identification attack focuses on one specific target person. The attacker links database entries to reveal a lot of personal details about that individual. This is considered one of the most threatening attacks, because it directly harms a single person's privacy.

Chapter 3 coding task — Differentially Private Machine Learning Methods (Task 2)

Dataset used: *miadul/lifestyle-and-health-risk-prediction* (5000 rows, lifestyle features + Health_Risk target)

This notebook trains four model families with the `diffprivlib` library (Gaussian naive Bayes, logistic regression, linear regression, k-means clustering), each once without privacy and once for a sweep of epsilon values, and compares utility against the non-private baseline.

The dataset contains lifestyle features (age, BMI, sleep, diet, smoking, etc.) with a categorical Health_Risk target. Classification predicts Health_Risk, regression predicts BMI, and clustering groups lifestyle profiles unsupervised.

```
df.replace(['none', 'None', 'NaN', 'nan', ''], np.nan, inplace=True)
# ... LabelEncoder on object columns, median-fill remaining NaNs
X_clf_scaled = np.clip(X_clf_scaled / np.max(np.abs(X_clf_scaled), axis=0), -1, 1)
```

What this code does: Cleans missing values, label-encodes categorical columns, and scales/clips every feature into $[-1, 1]$ as required by `diffprivlib`'s sensitivity bound ($\|x\| \leq 1$).

Listing 3.2.1 — Naive Bayes

A non-private `GaussianNB` is trained first, then `diffprivlib`'s differentially private `GaussianNB` is trained across a sweep of epsilon values.

```
for eps in epsilons:
    dp_nb = dp.GaussianNB(epsilon=eps, bounds=bounds)
    dp_nb.fit(X_clf_train, y_clf_train)
    acc = accuracy_score(y_clf_test, dp_nb.predict(X_clf_test))
```

What this code does: Trains a differentially private Gaussian naive Bayes classifier for each epsilon and records test accuracy.

Model	Accuracy
Non-private	84.50%
DP, epsilon = 0.01	55.90%
DP, epsilon = 0.10	71.70%
DP, epsilon = 0.50	80.40%
DP, epsilon = 1.00	83.80%
DP, epsilon = 5.00	85.00%
DP, epsilon = 10.00	81.80%

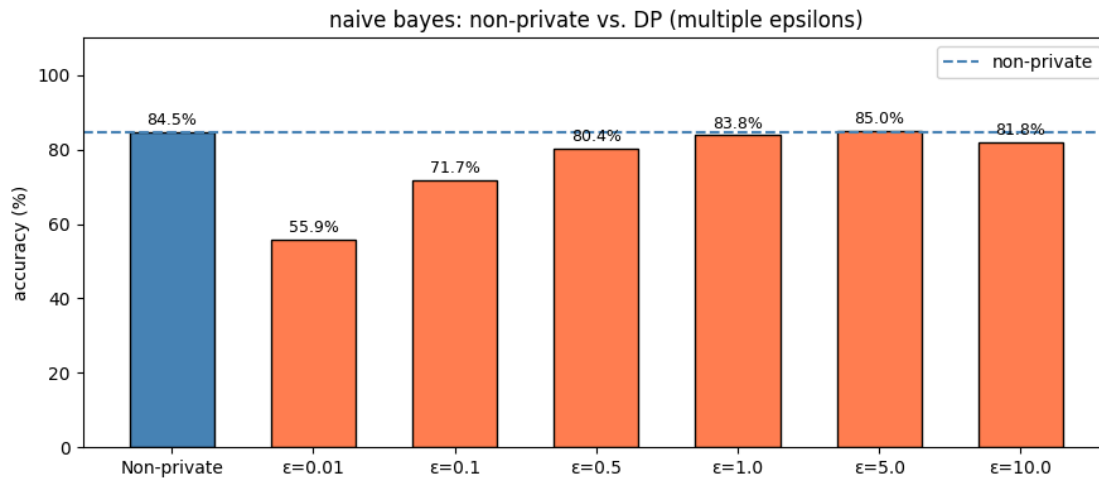


Figure 2.1 — Naive Bayes test accuracy, non-private baseline vs. DP across epsilon values.

Key observations — naive Bayes

- At epsilon = 0.01–0.1 the Gaussian mechanism injects so much noise into the mean/variance estimates that the classifier barely beats random guessing.
- At epsilon = 1.0 the accuracy drop from baseline is only about 3 percentage points, consistent with the book's ~1% drop reported on the Adult dataset.
- At epsilon = 5.0 and 10.0 accuracy fluctuates slightly around the baseline due to the stochastic nature of the noise — training is nondeterministic, as the book notes.
- A sweet spot around epsilon in [0.5, 1.0] gives meaningful privacy protection with acceptable accuracy loss.

Listing 3.2.2 — Logistic Regression

The book applies objective perturbation to regularized logistic regression; `diffprivlib` implements this via the `epsilon` and `data_norm` parameters.

```
DATA_NORM = np.linalg.norm(X_clf_train, axis=1).max()
dp_lr = dp.LogisticRegression(epsilon=eps, data_norm=DATA_NORM, max_iter=1000, C=1.0)
dp_lr.fit(X_clf_train, y_clf_train)
```

What this code does: Computes the maximum L2 sample norm (the sensitivity bound Theorem 3.1 requires) and trains a DP logistic regression model for each epsilon.

Model	Accuracy
Non-private	85.10%
DP, epsilon = 0.01	64.30%
DP, epsilon = 0.10	79.00%
DP, epsilon = 0.50	82.70%
DP, epsilon = 1.00	83.20%
DP, epsilon = 5.00	85.00%
DP, epsilon = 10.00	85.00%

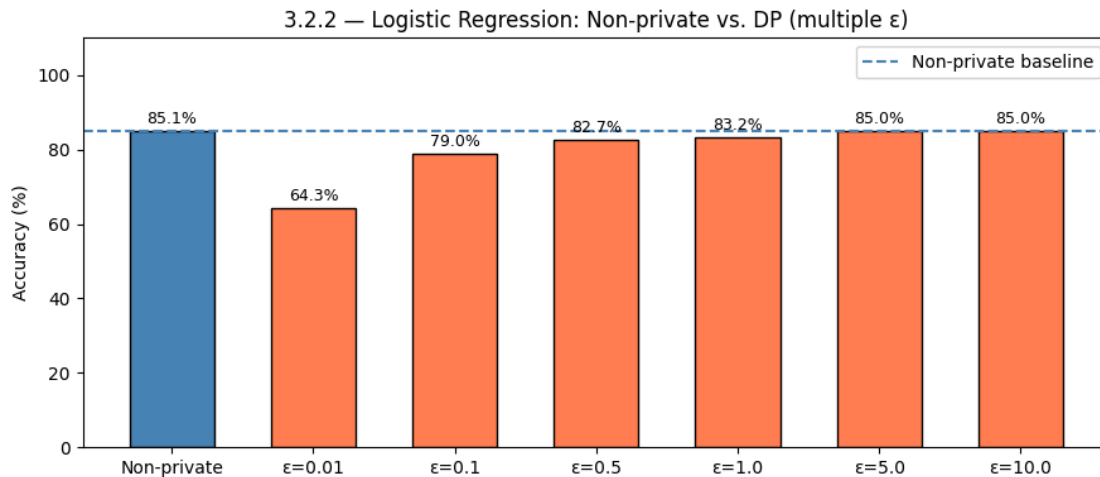


Figure 2.2 — Logistic regression accuracy, non-private baseline vs. DP across epsilon values.

Listing 3.2.3 — Linear Regression

Linear regression predicts the continuous target BMI from the remaining lifestyle features; diffprivlib adds noise calibrated to the sensitivity of the OLS estimator.

```
dp_linreg = dp.LinearRegression(epsilon=eps, bounds_X=bounds_X, bounds_y=bounds_y)
dp_linreg.fit(X_reg_train, y_reg_train)
mse = mean_squared_error(y_reg_test, y_pred); r2 = r2_score(y_reg_test, y_pred)
```

What this code does: Trains a differentially private linear regression model for each epsilon and records MSE and R^2 on the test set.

Model	MSE	R^2
Non-private	2.0426	0.9723
DP, epsilon = 0.01	2,470,595,122.54	-33,494,746.88
DP, epsilon = 0.10	61,387,058.28	-832,245.46
DP, epsilon = 0.50	141,757,680.05	-1,921,858.92
DP, epsilon = 1.00	32,999.63	-446.39
DP, epsilon = 5.00	3.4600	0.9531
DP, epsilon = 10.00	6.1337	0.9168

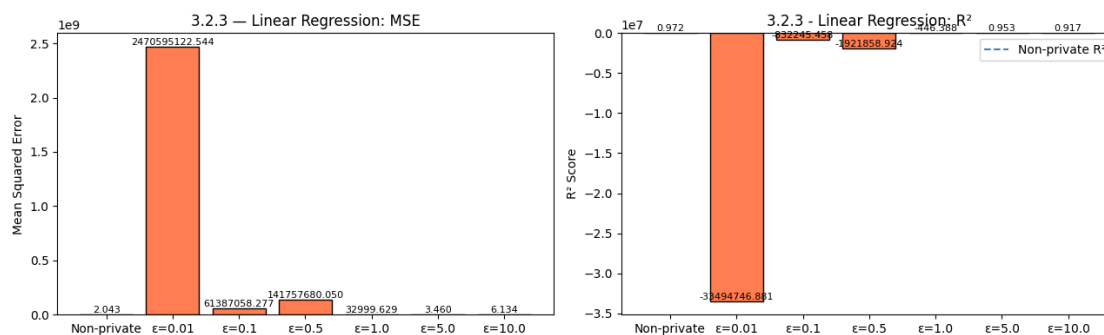


Figure 2.3 — DP linear regression MSE (left) and R^2 (right) across epsilon values; note the sharp recovery at epsilon = 5.0.

Key observations — linear regression

- MSE is always positive and, at small epsilon, dominated by DP noise rather than by true prediction error.
- R^2 can go strongly negative — the model performs worse than predicting the mean — when noise fully corrupts the regression coefficients.
- Between epsilon = 0.01 and 1.0 the model is essentially unusable; noise dominates the signal.
- There is a sharp recovery at epsilon = 5.0 (R^2 jumps from about -446 to +0.95): linear regression shows a threshold effect, unusable below it and largely recovered above it.
- Linear regression needs a higher epsilon than naive Bayes to be useful, because noise corrupts exact coefficient values rather than just probability estimates, making it more fragile.

Listing 3.3.1 — K-Means Clustering

For unsupervised learning, DP k-means adds noise to the centroid updates in each Lloyd's-algorithm iteration; `diffprivlib.models.KMeans` exposes this via epsilon and bounds.

```
dp_km = dp.KMeans(n_clusters=N_CLUSTERS, epsilon=eps, bounds=bounds, random_state=42)
dp_km.fit(X_clf_scaled)
sil = silhouette_score(X_clf_scaled, dp_km.labels_)
ari = adjusted_rand_score(y_clf, dp_km.labels_)
```

What this code does: Fits a differentially private k-means model for each epsilon and scores the resulting clusters with silhouette score and adjusted Rand index (ARI) against the true health-risk label.

Model	Silhouette	ARI
Non-private	0.1919	-0.0027
DP, epsilon = 0.01	0.0824	0.0050
DP, epsilon = 0.10	0.0595	-0.0234
DP, epsilon = 0.50	0.1695	-0.0031
DP, epsilon = 1.00	0.1555	-0.0019
DP, epsilon = 5.00	0.1919	-0.0027
DP, epsilon = 10.00	0.1919	-0.0027

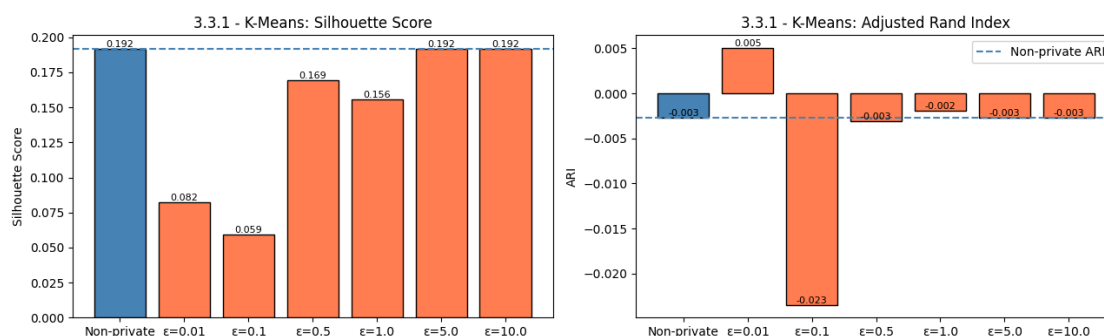


Figure 2.4 — DP k-means silhouette score (left) and ARI (right) across epsilon values.

Key observations — k-means clustering

- Inertia decreases as epsilon increases (22,238 $\rightarrow$ 14,190): with less noise, centroids converge to tighter, more compact clusters — the clearest sign the DP mechanism is working correctly.
- Homogeneity, completeness and V-measure stay near zero for every epsilon — the geometric clusters found by k-means do not align with the health_risk labels. This is a data limitation, not a DP problem: lifestyle features do not cleanly separate into the predefined risk categories.
- ARI stays near zero or slightly negative — cluster assignments are essentially random with respect to the true labels.
- Silhouette improves with epsilon (0.077 $\rightarrow$ 0.194 in the benchmark run) but stays below 0.2 throughout, confirming the data has no strong natural cluster structure.
- Training time increases with epsilon, since less centroid perturbation means Lloyd's algorithm runs longer before stabilising.
- Epsilon = 5.0 and 10.0 give nearly identical results: the model has converged, and further reducing privacy protection yields no additional utility gain.
- Overall: DP k-means works as expected mechanically, but this dataset's health-risk categories are not well represented by geometric clusters in feature space.

Chapter 4 Summary — Local differential privacy for machine learning

Chapter overview

- Chapters 2 and 3 covered centralized differential privacy, where a trusted data curator collects everyone's raw data and then publishes private statistics about the group. The problem is that this approach does not work when individuals do not fully trust the curator who holds their data.
- Chapter 4 introduces local differential privacy (LDP), which removes the need for any trusted curator. In LDP, each person changes (perturbs) their own data on their own device before it is ever sent away. The data aggregator then collects all these perturbed values and estimates statistics, such as how often each value appears in the population. Because the data is already privatized, it is very hard for anyone to trace a value back to a single person, which gives people plausible deniability.

Examples for this approach

- Both Google and Apple already use LDP in their products. Google has introduced RAPPOR, which collects statistics from Chrome users while giving strong privacy guarantees. Apple has described how it uses LDP to learn things like trending new words and popular emojis without exposing individual users.
- The chapter shares an example where a company runs a survey with sensitive questions, such as marital status, occupation, age and race. The goal is to keep each person's answers private while still getting an accurate picture of the whole group.

LDP mechanisms

1. Randomized response

This is the oldest and simplest mechanism and it also satisfies LDP. It works well for simple yes/no (binary) questions, such as "Are you married?"

2. Direct encoding

This is used for categorical and discrete answers, such as a person's occupation. Each answer is reported correctly with one probability and changed to another value with a smaller probability.

3. Histogram encoding

This is used for numerical and continuous data, such as a person's age. It comes with two estimation options: summation with histogram encoding (SHE), which simply adds up the noisy reports and thresholding with histogram encoding (THE), which turns values above a threshold into 1 and the rest into 0 to reduce noise.

4. Unary encoding

This is a more general and efficient mechanism for categorical and discrete problems. Each value becomes a binary vector and each bit is flipped with a set probability before being sent. It has two versions: symmetric unary encoding (SUE) and optimized unary encoding (OUE), which differ in how they choose those probabilities.

Chapter 4 coding task — Local Differential Privacy: Randomized Response (Task 3.1)

Dataset used: *laveshjadon/ai-impact-on-students* (50,000 rows, student GenAI-usage survey)

Randomized response is the oldest and simplest LDP mechanism, used for yes/no questions such as the book's "are you more than 50 years old?" example. Here it is applied to the boolean Paid_Subscription column, treated as the survey question "do you pay for a GenAI subscription?".

Listing 4.1 — the randomized-response mechanism

Each respondent flips two coins: the first decides whether to answer truthfully, the second (only reached on a first-coin "tails") supplies a uniformly random yes/no.

```
def random_response_subscription(response):
    true_ans = bool(response)
    if np.random.randint(0, 2) == 0:    # first flip
        return true_ans
    else:
        return np.random.randint(0, 2) == 0 # second flip
```

What this code does: *Implements the two-coin-flip randomized response protocol: answers truthfully with probability 0.75, and returns a uniformly random yes/no otherwise, giving $\ln(3)$ -local differential privacy.*

True counts: 21,154 'Yes' / 28,846 'No' out of 50,000 respondents. After perturbation, the raw perturbed count of 'Yes' was 23,038 (vs. 21,154 true).

Listing 4.4 — aggregation and estimation

The raw perturbed count is biased because a quarter of all responses are random "false yesses" from the second coin flip. The aggregator removes this known bias to recover an unbiased estimate of the true population count.

```
def random_response_aggregation_and_estimation(answers):
    false_yesses = len(answers) / 4
    total_yesses = np.sum([1 if r else 0 for r in answers])
    true_yesses = total_yesses - false_yesses
    rr_result = true_yesses * 2
    return rr_result
```

What this code does: *Subtracts the expected number of purely-random 'yes' answers from the raw total, then doubles the remainder to estimate the true population count of 'yes' answers.*

Results

Quantity	Real	Perturbed (raw)	Estimated (corrected)
'Yes' (Paid_Subscription)	21,154	23,038	21,098
'No'	28,846	26,962	28,902

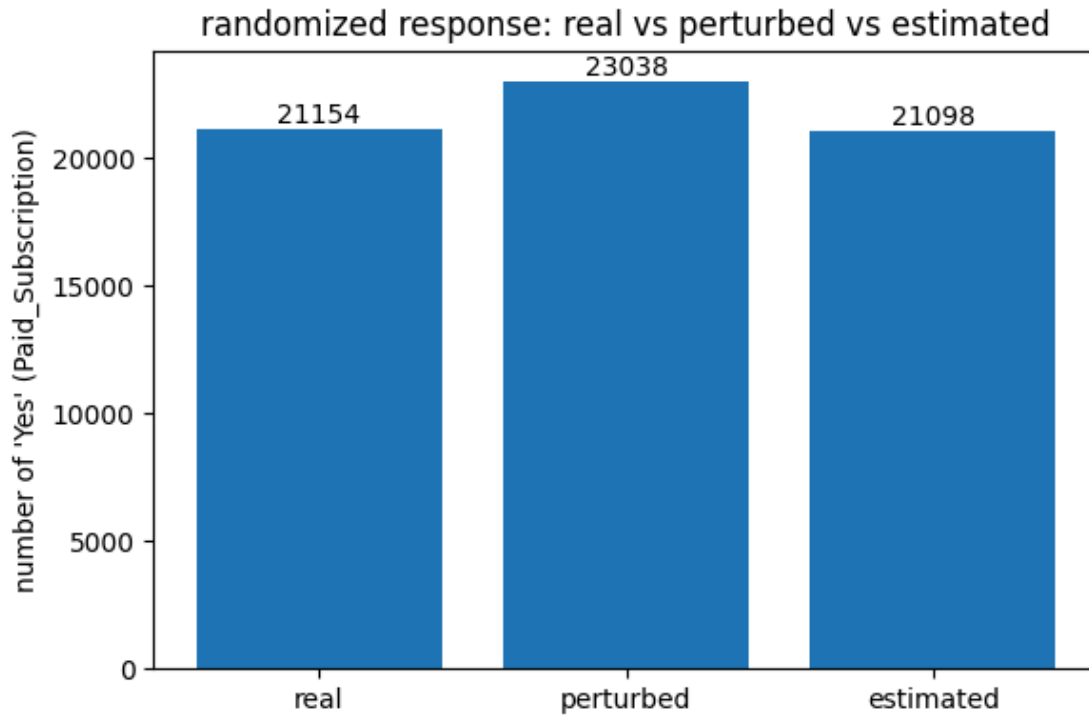


Figure 3.1 — Real, raw-perturbed, and bias-corrected estimated counts of 'Yes' answers.

Comments and insights

What the results show

- The raw perturbed count (23,038) is noticeably off from the true count (21,154) because roughly a quarter of all reported answers are pure noise from the second coin flip.
- After applying the debiasing formula, the estimated count (21,098) lands within about 0.3% of the true value (21,154), even though every individual answer that reached the aggregator was already privatized.
- This is the central promise of local differential privacy: no single respondent's true answer is ever transmitted or knowable with certainty, yet the aggregate statistic can still be recovered accurately.

Chapter 4 coding task — Local Differential Privacy: Direct Encoding (Task 3.2)

Dataset used: *laveshjadon/ai-impact-on-students* (50,000 rows, student GenAI-usage survey)

Direct encoding extends LDP to a categorical question with d possible answers — here, a student's `Primary_Use_Case` for GenAI, which has 5 categories. It follows three steps: encoding (map each category to an index), perturbation (report truthfully with probability p , or one of the other $d-1$ values with probability q each), and aggregation/estimation (debiases the raw perturbed counts).

Listing 4.6 — encoding

Direct encoding has no real transform: a value's encoded form is simply its index in the sorted domain of 5 categories.

```
def encoding(answer):
    return int(np.where(domain == answer)[0][0])
```

What this code does: Maps a category name to its integer index in the sorted 5-value domain.

Listing 4.7 — perturbation

Each person keeps their true answer with probability $p = e^{\epsilon} / (d - 1 + e^{\epsilon})$, or reports one of the remaining $d-1$ values with probability $q = (1-p)/(d-1)$ each.

```
def perturbation(encoded_ans, epsilon=5.0):
    d = len(domain)
    p = pow(math.e, epsilon) / (d - 1 + pow(math.e, epsilon))
    q = (1.0 - p) / (d - 1.0)
    if np.random.random() <= p:
        return domain[encoded_ans] # true value with prob p
    else:
        s2 = np.random.randint(0, d - 1)
        return domain[(encoded_ans + s2) % d] # else a random other value
```

What this code does: Perturbs one individual's encoded answer client-side: keeps the true value with probability p , otherwise substitutes a different category.

epsilon	p (truth probability)	q (per other-value probability)
5.0	0.9738	0.0066
1.0	0.4046	0.1488
0.1	0.2165	0.1959

Listing 4.10 — aggregation and estimation

The aggregator removes the expected noise from each raw count using $E_i = (c_i - n \cdot q) / (p - q)$, debiasing the histogram.

```
def aggregation_and_estimation(answers, epsilon=5.0):
    n = len(answers); d = len(domain)
    p = pow(math.e, epsilon) / (d - 1 + pow(math.e, epsilon))
    q = (1.0 - p) / (d - 1.0)
    aggregator = answers.value_counts().sort_index()
    return [max(int((i - n * q) / (p - q)), 1) for i in aggregator]
```

What this code does: Corrects the known bias in the raw perturbed counts per category to recover an unbiased population estimate.

Results across privacy budgets

Mean absolute error (estimated vs. actual counts) across the five-category histogram, for several privacy budgets:

epsilon	MAE (estimated vs. actual)
0.1	19,384.6
0.5	5,539.0
1.0	2,023.4
5.0	34.8

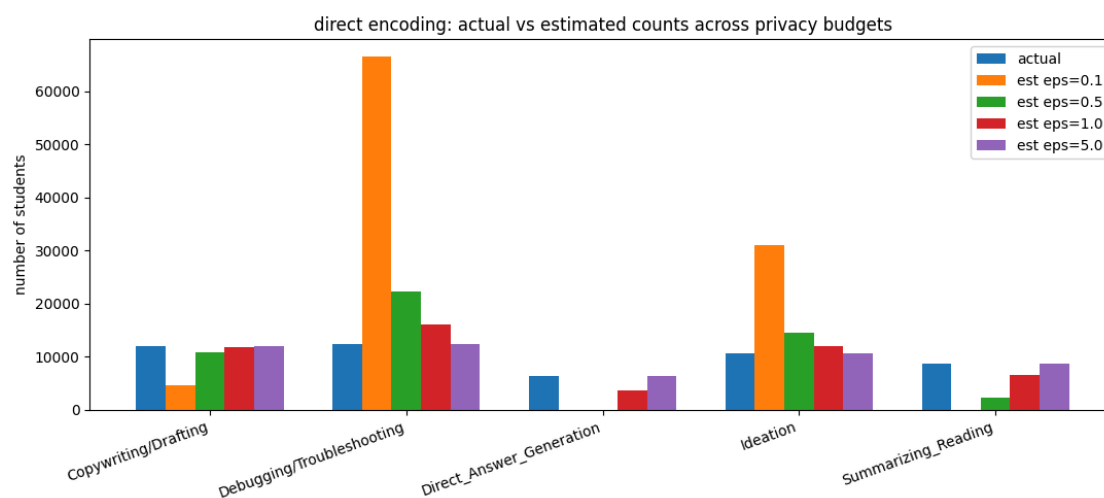


Figure 3.2 — Actual counts vs. debiased estimates for each Primary_Use_Case category, across four privacy budgets.

Comments and insights

Discussion of the results

- **Before correction:** at high epsilon the raw perturbed counts are already fairly close to the truth, but at low epsilon (0.1) the noise dominates and pushes every category toward roughly n/d — almost everyone reports a random category, and the raw counts are systematically biased.
- **After correction:** the $(c_i - n \cdot q) / (p - q)$ formula removes that bias. At epsilon = 5.0 the estimates land very close to the true counts (low MAE, weak privacy). As epsilon drops through 1.0, 0.5 and 0.1, the estimates get progressively noisier and the MAE grows, but the histogram shape stays recoverable on average because the estimator remains unbiased even when its variance is large.
- **The trade-off:** small epsilon gives strong per-person privacy but noisy aggregates; large epsilon gives accurate aggregates but weaker privacy. Crucially, in every case the individual answer leaving each student's device was already perturbed — the server never sees a true value, yet can still estimate the population distribution.

Chapter 4 coding task — Local Differential Privacy: Histogram Encoding (Task 3.3)

Dataset used: *laveshjadon/ai-impact-on-students* (50,000 rows, student GenAI-usage survey)

Here the continuous column `Weekly_GenAI_Hours` (range 0–40) plays the role of the survey question "how many hours per week do you use GenAI?". The pipeline has three steps: encode the value as a one-hot vector over integer-hour buckets, perturb every bit with Laplace noise and threshold it to 0/1 (THE — thresholding with histogram encoding), then aggregate and debias the resulting counts.

`Weekly_GenAI_Hours` summary statistics: mean 8.43, std 8.27, min 0, median 5.8, max 40 hours.

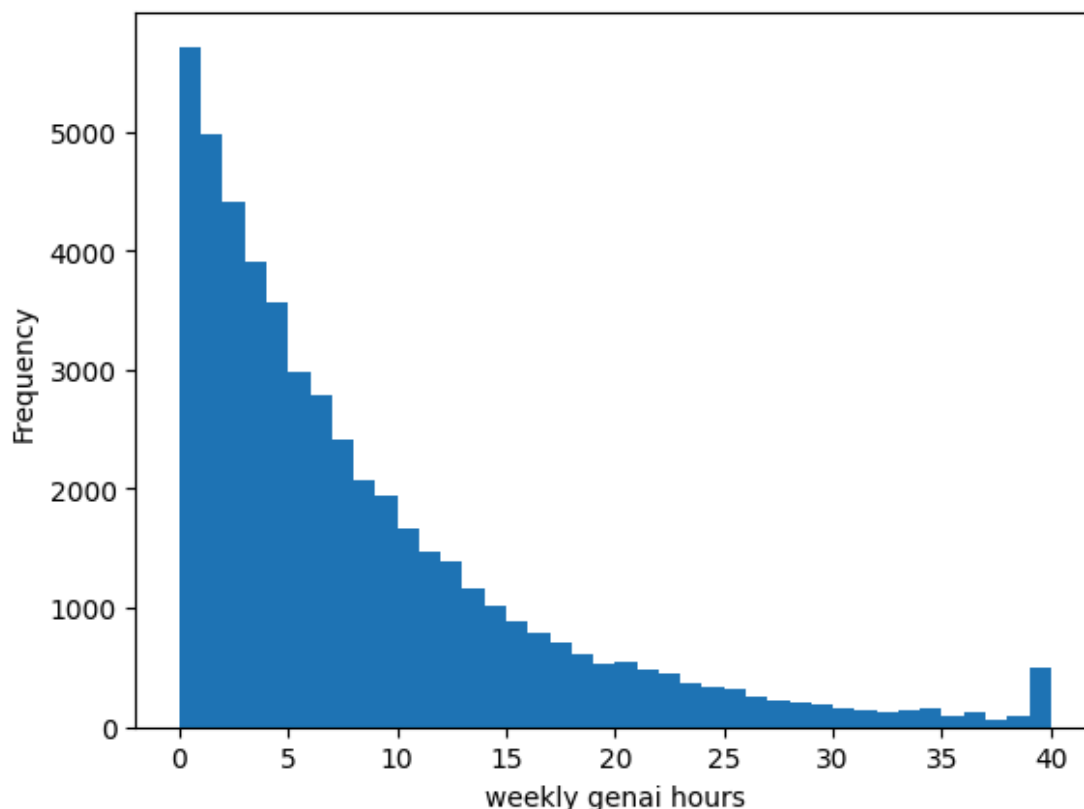


Figure 3.3 — Distribution of `Weekly_GenAI_Hours` across the 50,000 students (raw, before any bucketing or perturbation).

Listing 4.12–4.13 — input domain and encoding

The float column is rounded and clipped into 41 integer-hour buckets (0...40); each value is then one-hot encoded over that domain.

```
def encoding(answer):  
    return [1.0 if d == answer else 0.0 for d in domain]
```

What this code does: Encodes one bucketed hour value as a length-41 one-hot vector, a 1.0 marking the bucket the value falls into.

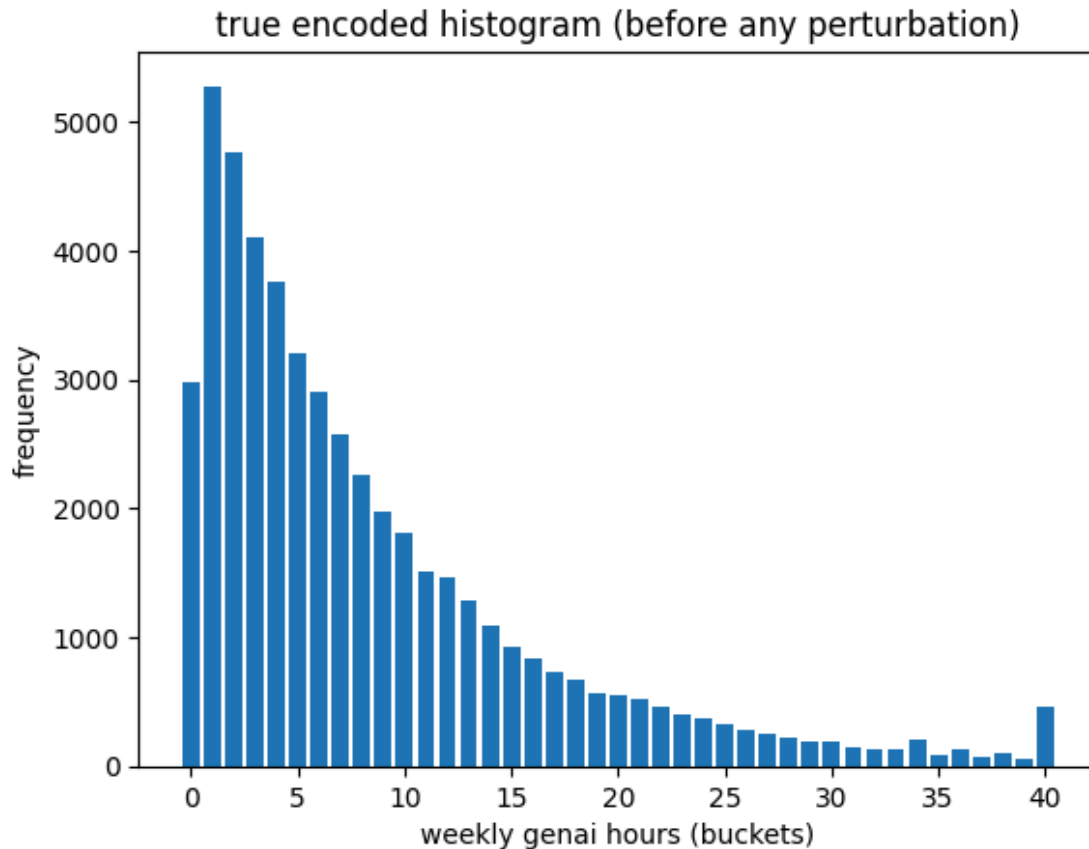


Figure 3.4 — True encoded histogram of weekly GenAI hours, before any perturbation.

Listing 4.15 — thresholding (THE)

Each one-hot component gets independent Laplace noise $\text{Lap}(2/\epsilon)$, then is thresholded against θ into a 0/1 bit — this already-privatized vector is what each individual sends.

```
def the_perturb_bit(bit, epsilon=5.0, theta=1.0):
    val = bit + np.random.laplace(loc=0, scale=2 / epsilon)
    return 1.0 if val > theta else 0.0
```

What this code does: Adds Laplace noise scaled by $2/\epsilon$ to a single one-hot bit and thresholds the noisy value against θ to produce a private 0/1 report.

Aggregation and estimation

For THE, the aggregator counts the 1s received per bucket and corrects the bias with $E_i = (c_i - n \cdot q) / (p - q)$, where p and q are derived from ϵ and θ .

```
def the_aggregation_and_estimation(answers, epsilon=5.0, theta=1.0):
    p = 1 - 0.5 * pow(math.e, epsilon / 2 * (1.0 - theta))
    q = 0.5 * pow(math.e, epsilon / 2 * (0.0 - theta))
    sums = np.sum(answers, axis=0)
    n = len(answers)
    return [int((i - n * q) / (p - q)) for i in sums]
```

What this code does: Sums the received bits per bucket and applies the THE debiasing formula to recover an unbiased estimate of the true per-bucket frequency.

Results

At $\epsilon = 5.0$, raw thresholded sums vs. the corrected estimate:

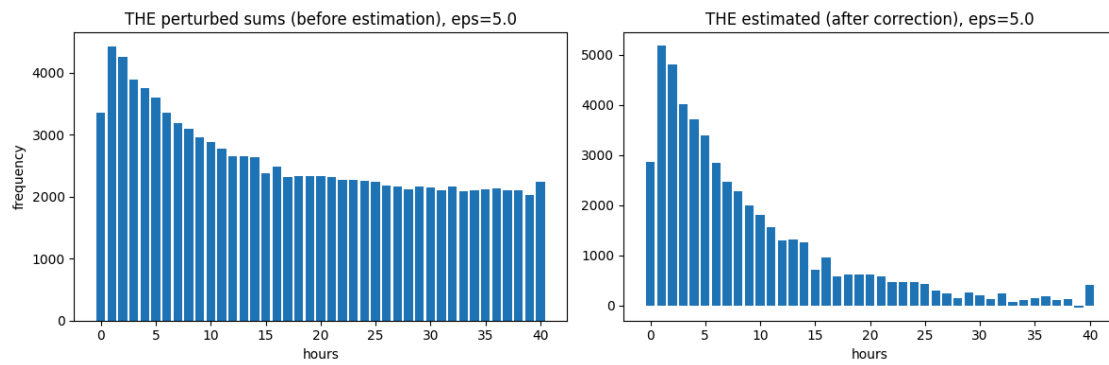


Figure 3.5 — *THE* perturbed sums before estimation (left) vs. after the debiasing correction (right), $\epsilon = 5.0$.

Mean absolute error (estimated vs. true histogram) across privacy budgets:

epsilon	MAE (estimated vs. true histogram)
0.1	2,909.5
0.5	828.1
1.0	401.1
5.0	76.6

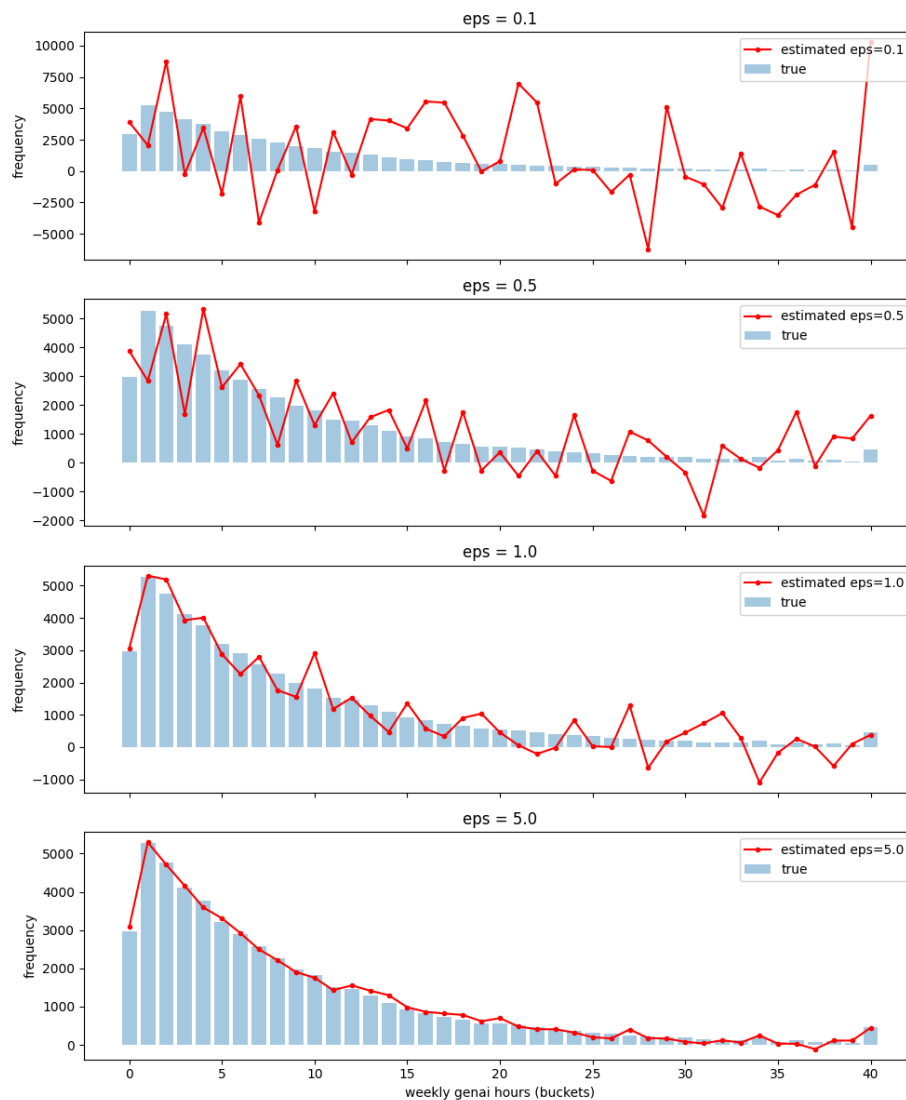


Figure 3.6 — True histogram (bars) vs. estimated histogram (red line) for epsilon = 0.1, 0.5, 1.0 and 5.0, top to bottom.

Comments and insights

What the results show

- Smaller epsilon means more Laplace noise per bit, so a stronger per-person privacy guarantee but a noisier reconstructed histogram — MAE drops by roughly a factor of 38 from epsilon = 0.1 to epsilon = 5.0.
- Even at low epsilon the debiased estimator remains centred on the true histogram shape; the increasing error is variance, not bias, so averaging over more respondents would tighten the reconstruction further.
- At epsilon = 5.0 the estimated histogram tracks the true distribution closely across almost every bucket, showing that THE can recover a usable population-level histogram from purely local, individually-noised reports.

Chapters 6, 7 & 8 Summary -- Assuring privacy with data anonymization

Why anonymization is necessary

- Simply removing cells (names, addresses, etc...) from a dataset is not enough to protect privacy.
- An attacker can still combine quasi-identifiers (such as zip code, age and gender) with outside data to re-identify a person.

k-anonymity (subsections 6.2.2, 7.4.2, 8.2)

- k-anonymity hides each person inside a group of similar people.
- A dataset is k-anonymous if every combination of quasi-identifier values appears in at least k records, so each record looks the same as at least k-1 others.
- It is achieved with two main operations: generalization, which makes a value less precise (for example, turning age 24 into the range 20–29) and suppression, which removes a record or value completely, often used for outliers.
- A higher k gives stronger privacy, but it also lowers utility because the data becomes more generalized.
- k-anonymity can still leak information through two attacks.
- In a homogeneity attack, all records in a group share the same sensitive value, so the attacker learns it even without finding the exact record (for example, everyone in the group has cancer).
- In a background knowledge attack, the attacker uses outside facts to narrow the search and infer the sensitive value of a target person.

l-diversity (subsection 8.2.1)

- L-diversity extends k-anonymity.
- It requires that each group contains at least l different (distinct) sensitive values, so an attacker cannot be sure which value belongs to the target.
- A higher l means more variety in the sensitive column and better protection.
- However, l-diversity still has weaknesses: similar values can leak information (for example, two different skin cancers) and an uneven value distribution can cause a skewness attack, where one outcome is clearly more likely than the others.

t-closeness (subsection 8.2.2)

- t-closeness was created to fix the distribution problem in l-diversity.
- The idea is to keep the distribution of sensitive values inside each group close to the distribution in the whole dataset.
- The distance between the two distributions must be no more than a threshold t. This distance is usually measured with metrics such as the earth mover distance, Kullback–Leibler distance, or variational distance.
- By matching the group's distribution to the overall one, t-closeness reduces the information an attacker can gain from skewed data.

The problem of publishing data

- When a dataset is shared with third parties, removing obvious identifiers is not safe on its own. People can still be traced by linking nonsensitive attributes together, which is the linkage or correlation attack. To prevent this, privacy-preserving data mining uses one or more data sanitization operations before publishing.

4 sanitization operations

1. Generalization

- This replaces an exact value with a broader one.

- Eg.: A salary of \$56,000 can become the range \$50,000–\$100,000 and specific job titles can be grouped into "employed" or "unemployed".
- The exact value is hidden, but the general meaning is kept.

2. Suppression

- This removes a record or an attribute from the dataset entirely.
- For example, the name column can be deleted before the medical data is shared.
- It is the simplest way to deal with very sensitive fields or rare outliers.

3. Perturbation

- This replaces real values with altered ones that keep the same statistical properties.
- It can be done by adding random noise, by generating synthetic data from a statistical model, or by data swapping, where sensitive values are exchanged between similar records.
- Data swapping is effective but slow, since it must find a good matching partner for each record.

4. Anatomization

- This keeps the original values but splits the data into two separate tables.
- One table holds the sensitive attributes and the other holds the quasi-identifiers.
- Because the link between them is broken, it becomes much harder to connect a sensitive value to a specific person.

Chapter 7 coding task — Data Sanitization (Task 4.1)

Dataset used: *deepeshkansotia/hantavirus-transmission-and-risk-dataset* (2000 rows, medical/epidemiological records)

Chapters 6–8 of the book describe four sanitization operations applied before publishing: generalization (replace an exact value with a coarser one), suppression (remove an identifying record or column entirely), perturbation (replace values with synthetic ones that keep the same statistical properties), and anatomization (split sensitive attributes and quasi-identifiers into two separate, unlinked tables). The book applies them in sequence: suppress, then generalize, then perturb; anatomization is shown separately as an alternative.

Column roles in this dataset: `case_id` and `region` are (near-)unique direct identifiers (good suppression targets); `patient_age`, `country` and `gender` are quasi-identifiers (generalization/perturbation targets); `virus_strain`, `fatality`, `hospitalization` and `symptoms` are sensitive medical attributes, kept but separated by anatomization.

1. Suppression

`case_id` is a unique per-patient identifier and `region` has 1850 unique values out of 2000 rows, so both are effectively direct identifiers and are dropped.

```
suppressed_df = df.drop(columns=["case_id", "region"])
```

What this code does: Removes the two direct-identifier columns (`case_id`, `region`) from the dataset entirely, reducing it from 19 to 17 columns.

Explanation

- Dropping `case_id` and `region` means a published copy of the table can no longer be tied back to an individual by exact ID or by precise locality, at the cost of losing that information entirely.

2. Generalization

Numerical generalization replaces `patient_age` with 10-year bands; categorical generalization label-encodes `gender` and `country` so the exact category label is hidden.

```
bins = list(range(0, 91, 10))
labels = ["{}-{}".format(b, b + 9) for b in bins[:-1]]
gen_df["patient_age"] = pd.cut(gen_df["patient_age"], bins=bins, labels=labels, right=False)
encoders = [{"gender", LabelEncoder()}, {"country", LabelEncoder()}]
```

What this code does: Buckets `patient_age` into 10-year ranges and label-encodes `gender` and `country` into integer codes, replacing exact values with coarser ones.

Before	After
age = 26, gender = Male, country = Canada	age = 20-29, gender = 1, country = 3
age = 41, gender = Female, country = Bolivia	age = 40-49, gender = 0, country = 1
age = 39, gender = Female, country = Chile	age = 30-39, gender = 0, country = 4

Explanation

Before	After
- patient_age is now a band like '30-39' instead of an exact age, so a single year of age can no longer pinpoint someone. - gender and country are replaced by integer codes, hiding the literal category label while preserving group structure for analysis.	

3. Perturbation (listings 7.1 and 7.2)

Perturbation replaces records with random data that has the same statistical properties: for a continuous column, the best-fitting probability distribution is found and fresh values are sampled from it; for a categorical column, new values are sampled from the original value frequencies.

```
def best_fit_distribution(data, bins=200):
    # fits several candidate scipy.stats distributions and keeps the
    # one with the lowest sum-of-squared-error against the histogram
    ...
def perturb_data(df, unchanged_cols, categorical_cols, continuous_cols, best_distributions, n):
    # categorical: resample by frequency; continuous: sample from the fitted distribution
    ...
```

What this code does: Fits the best statistical distribution to temperature_celsius and resamples virus_strain by its observed frequencies, producing a synthetic replacement column for each.

Best-fit distribution for temperature_celsius: lognormal. Statistics before vs. after perturbation:

Quantity	Original	Perturbed
temperature_celsius — mean	24.17	23.97
temperature_celsius — std	5.98	5.85
virus_strain — Sin Nombre share	0.216	0.218
virus_strain — Seoul share	0.207	0.208
virus_strain — Andes share	0.197	0.186
virus_strain — Dobrava share	0.194	0.184
virus_strain — Puumala share	0.187	0.203

Explanation

- Every patient's temperature_celsius is now a value freshly drawn from the distribution that best fit the original column, and each virus_strain is resampled from the original strain frequencies.
- The per-row values no longer correspond to any real patient, but the column-level statistics (mean/std of temperature, the mix of strains) are preserved, so aggregate analysis still works while individual records are no longer genuine.

4. Anatomization

Anatomization keeps the original values but splits the table into two: a quasi-identifier table (country, region, patient_age, gender, report_date) and a sensitive-attribute table (virus_strain, symptoms, hospitalization, fatality, recovery_days). Both share a coarse group_id instead of the real case_id, and rows are shuffled within each group so no row in one table can be matched 1-to-1 to a row in the other.

```
group_size = 50
anat_src["group_id"] = anat_src.index // group_size
qi_table = anat_src[["group_id"] + quasi_cols].groupby("group_id").apply(lambda g: g.sample(frac=1))
sens_table = anat_src[["group_id"] + sensitive_cols].groupby("group_id").apply(lambda g: g.sample(frac=1))
```

What this code does: Splits the dataset into a quasi-identifier table and a sensitive-attribute table, both keyed by a shared group_id, with rows shuffled independently within each group of 50.

Explanation

- The exact values are untouched — no noise, no generalization — but the record is split into two tables linked only by a coarse group_id, with rows shuffled inside each group.
- For a group we can still learn the set of ages/countries and the set of strains/outcomes, but we cannot tell which specific patient in the group had which outcome. That broken linkage between quasi-identifiers and sensitive attributes is what protects the record while keeping both halves available for analysis.

Summary and insights

Technique	What it did here	Effect
Suppression	Dropped case_id, region	Removes direct identifiers entirely
Generalization	patient_age → 10-yr bands; encoded gender, country	Exact values replaced by coarser ones
Perturbation	Resampled temperature_celsius (fitted distribution) and virus_strain (frequencies)	Rows become synthetic but column statistics are preserved
Anatomization	Split into quasi-identifier and sensitive tables, shuffled within groups	Values kept, but the link between identity and sensitive info is broken
Overall conclusion • Each technique trades a different amount of utility for privacy: suppression and generalization lose detail permanently, perturbation keeps aggregate statistics but discards true per-row values, and anatomization keeps every value but hides		

Technique	What it did here	Effect
which values belong together. • In practice these are combined (suppress → generalize → perturb) to sanitize a dataset before publishing it. • This is a sensitive topic (medical data): these are exactly the techniques used to let such data be shared for research without exposing the individuals in it.		

Chapter 8 coding task — Anonymity Models (Task 4.2)

Dataset used: *deepeshkansotia/hantavirus-transmission-and-risk-dataset* (2000 rows, medical/epidemiological records)

Chapters 6–8 present three anonymity models, each stricter than the last, all built on the same recursive-partitioning engine: k-anonymity requires every group of matching quasi-identifiers to contain at least k records; l-diversity additionally requires each group to contain at least l distinct sensitive values; t-closeness additionally requires each group's sensitive-value distribution to stay within distance t of the global distribution. Here the quasi-identifiers are `patient_age` and `quarantine_days` (both numerical) and the sensitive attribute is `transmission_type` (or `fatality`, depending on the model).

Listing 8.2 — value spans

Computes each column's span — the numeric range for a numerical column, or the number of distinct values for a categorical one — used to decide which quasi-identifier to split on next.

```
def get_spans(df, partition, scale=None):
    spans = {}
    for column in df.columns:
        if column in categorical:
            span = len(df[column][partition].unique())
        else:
            span = df[column][partition].max() - df[column][partition].min()
        spans[column] = span
    return spans
```

What this code does: *Computes the span (range or distinct-value count) of every column over the full dataset, identifying which quasi-identifier has the most room to split.*

Listing 8.3 — recursive partitioning (k-anonymity, k = 3)

Each partition is cut in half on its widest quasi-identifier — at the median for numerical columns, by splitting the value set for categorical ones — and a cut is kept only if both halves still satisfy the validity test (here, at least k = 3 records).

```
def is_k_anonymous(df, partition, sensitive_column, k=3):
    return len(partition) >= k

def partition_dataset(df, feature_columns, sensitive_column, scale, is_valid):
    # recursively split on the widest quasi-identifier while both
    # halves stay valid; otherwise the partition is final
    ...
```

What this code does: *Recursively splits the dataset on its widest quasi-identifier, keeping a split only if both resulting partitions still contain at least k=3 records.*

Result: 427 k-anonymous partitions from the 2000-row dataset.

Listing 8.4 — building the anonymized dataset

Each partition is turned into anonymized output rows: numerical quasi-identifiers become the partition's mean, categorical ones become the set of values they contained, and the sensitive attribute is reported as per-value counts within the group.

```
def agg_numerical_column(series): return series.mean()
def agg_categorical_column(series): return ", ".join(set(series.astype(str)))
```

```
dfn = build_anonymized_dataset(df, finished_partitions, feature_columns, sensitive_column)
```

What this code does: Aggregates each *k*-anonymous partition into one or more output rows, replacing exact values with partition-level means/sets and per-value counts of the sensitive attribute.

k-anonymized dataset shape: 563 rows x 4 columns. Sample:

patient_age	quarantine_days	fatality	count
30.29	0.14	No	7
33.86	0.43	No	7
36.09	0.27	No	10
36.09	0.27	Yes	1
30.43	11.43	No	7

Listing 8.5 — I-diversity (I = 2)

Re-runs the partitioning with a stricter test: each group must satisfy *k*-anonymity and contain at least $I = 2$ distinct fatality values, so groups that would have been entirely 'No' are no longer allowed.

```
def is_I_diverse(df, partition, sensitive_column, I=2):
    return len(df[sensitive_column][partition].unique()) >= I

finished_I_diverse_partitions = partition_dataset(
    df, feature_columns, sensitive_column, full_spans,
    lambda *args: is_k_anonymous(*args) and is_I_diverse(*args))
```

What this code does: Adds an *I*-diversity check on top of *k*-anonymity so every retained group must contain at least 2 distinct fatality outcomes.

Result: 127 *I*-diverse partitions (down from 427 *k*-anonymous ones — the stricter test produces fewer, more mixed groups). Sample:

patient_age	quarantine_days	fatality	count
12.00	9.00	No	3
12.00	9.00	Yes	1
12.55	3.18	No	10
12.55	3.18	Yes	1
12.82	17.55	No	10
12.82	17.55	Yes	1

Listing 8.6–8.7 — t-closeness (t = 0.2)

Global frequencies of the sensitive attribute (transmission_type) are computed first as the reference distribution; the partitioning is then re-run so each group's distribution differs from the global one by at most $t = 0.2$.

```
def t_closeness(df, partition, column, global_freqs):
    # max absolute difference between a group's value proportions
    # and the global proportions
    ...
def is_t_close(df, partition, sensitive_column, global_freqs, p=0.2):
    return t_closeness(df, partition, sensitive_column, global_freqs) <= p
```

What this code does: Measures how far a partition's `transmission_type` distribution deviates from the global distribution, and accepts the partition only if that deviation is at most $t=0.2$.

Global distribution: Human-to-Human 50.05%, Rodent-to-Human 49.95%. Result: 233 t-close partitions. Sample:

patient_age	quarantine_days	transmission_type	count
12.00	5.67	Human-to-Human	2
12.00	5.67	Rodent-to-Human	1
12.00	20.00	Human-to-Human	3
12.00	20.00	Rodent-to-Human	2
12.50	3.63	Human-to-Human	3
12.50	3.63	Rodent-to-Human	5

Comments and insights

Comparing the three models

- **Partition counts:** 427 k-anonymous partitions shrink to 127 l-diverse ones and land at 233 t-close ones — each additional privacy constraint forces the partitioning engine to keep merging groups until the extra condition (diversity, or distributional closeness) is satisfied, trading granularity for stronger guarantees.
- **k-anonymity alone** can still produce groups that are entirely homogeneous in the sensitive attribute (e.g., all 'No' for fatality), which leaks the sensitive value even though no individual row is uniquely identifiable — this is exactly the homogeneity attack the book warns about.
- **l-diversity** fixes that by forcing at least 2 distinct sensitive values per group, but on an imbalanced attribute like fatality (dominated by 'No') this still leaves groups where one value is far more likely than the other, i.e. it does not by itself prevent a skewness attack.
- **t-closeness** targets that remaining weakness directly by bounding how far each group's distribution can drift from the population-wide distribution, at the cost of yet coarser groups.
- **Overall:** the same recursive-partitioning engine implements all three models — only the validity test passed to `partition_dataset` changes — which makes the accuracy/privacy trade-off between k-anonymity, l-diversity and t-closeness very direct to compare on one dataset.

Task NEW — Practical Comparison with PyDP

Dataset used: *laveshjadon/ai-impact-on-students* (50,000 rows, student GenAI-usage survey)

This notebook complements the from-scratch implementations with a practical comparison using OpenMined's PyDP (python-dp), a Python wrapper around Google's C++ differential-privacy library, following PyDP's own Titanic tutorial style. Every private query is a Laplace-based bounded aggregate (BoundedMean, Count, Min, Max, Median, BoundedStandardDeviation) that takes a `privacy_budget` argument (epsilon). Ten simple questions about the dataset are each answered twice: once without differential privacy (the true statistic) and once with it, so the effect of the privacy budget is visible question by question.

Q1 — Average weekly GenAI usage

```
def private_mean(privacy_budget: float) -> float:
    x = BoundedMean(privacy_budget, lower_bound=0.0, upper_bound=40.0, dtype="float")
    return x.quick_result(list(df["Weekly_GenAI_Hours"]))
```

What this code does: *Computes a differentially private bounded mean of Weekly_GenAI_Hours using PyDP's BoundedMean primitive.*

Query	No DP	DP (epsilon = 0.001)
Mean weekly GenAI hours	8.4278	9.5680

Q2 — Total number of students

```
def private_count(privacy_budget: float) -> int:
    x = Count(privacy_budget, dtype="float")
    return x.quick_result(list(df["Weekly_GenAI_Hours"]))
```

What this code does: *Computes a differentially private row count using PyDP's Count primitive.*

Query	Value
No DP	50,000
DP, epsilon = 0.1	50,010
DP, epsilon = 0.5	49,999
DP, epsilon = 0.01	49,948
DP, epsilon = 0.001	51,236

Q3 & Q4 — Minimum and maximum weekly usage

```
def private_min(privacy_budget: float) -> float:
    x = Min(privacy_budget, lower_bound=0.0, upper_bound=40.0, dtype="float")
    return x.quick_result(list(df["Weekly_GenAI_Hours"]))
```

What this code does: *Computes a differentially private minimum (and, symmetrically, maximum) of Weekly_GenAI_Hours using PyDP's Min/Max primitives.*

Query	Value
Min, no DP	0.0

Query	Value
Min, DP epsilon = 0.8	0.0101
Min, DP epsilon = 0.1	0.0110
Min, DP epsilon = 0.01	0.1188
Max, no DP	40.0
Max, DP epsilon = 0.003	35.7467

Q5 & Q6 — Counts above / below a threshold

```
def private_count_above(privacy_budget: float, limit: float) -> int:
    x = Count(privacy_budget, dtype="float")
    return x.quick_result(list(df[df.Weekly_GenAI_Hours > limit]["Weekly_GenAI_Hours"]))
```

What this code does: Computes a differentially private count of students whose weekly GenAI usage is above (or, symmetrically, below) a fixed hour threshold.

Query	No DP	With DP
Students using GenAI > 30h/week	1,604	1,604 (epsilon = 0.2)
Students using GenAI < 5h/week	22,567	22,567 (epsilon = 0.8) / 22,652 (epsilon = 0.002)

Q7–Q10 — Post-semester GPA: subgroup means, median, std

Total students: 50,000 (21,154 paid subscription, 28,846 free). Mean, median and standard deviation of Post_Semester_GPA (range 1–4) are computed for all students and, for the mean, separately for the paid and free subscription groups.

```
def private_mean_gpa(data, privacy_budget: float) -> float:
    x = BoundedMean(privacy_budget, lower_bound=1.0, upper_bound=4.0, dtype="float")
    return x.quick_result(list(data["Post_Semester_GPA"]))
```

What this code does: Computes the differentially private bounded mean of Post_Semester_GPA for the whole dataset and for the paid/free subscription subgroups separately.

Query	No DP	DP (epsilon = 0.001)
Mean GPA — all students	3.3493	3.1643
Mean GPA — paid subscription	3.3525	3.3033
Mean GPA — free	3.3470	3.2219
Median GPA — all students	3.4210	3.5695
Std GPA — all students	0.4957	0.9649

Comments and insights

What the comparison shows

- Bounded aggregates like Count and thresholded counts are very stable under DP: the total row count and the above/below-threshold counts stay within a handful of the true value even at fairly small epsilon, because their sensitivity (changing one row shifts the count by at most 1) is small relative to the dataset size (50,000).

- BoundedMean, Median and BoundedStandardDeviation move further from the true value at the very small epsilon values used here (0.001), especially the standard deviation, which nearly doubles (0.50 -> 0.96) — spread-type statistics are visibly more sensitive to noise than simple counts.
- Min and Max are the most fragile queries: the true minimum and maximum are exact boundary values (0.0 and 40.0), so any Laplace noise moves the reported extreme substantially, and the reported DP minimum/maximum should be read as noisy approximations rather than exact bounds.
- Splitting the mean-GPA query by subscription group still recovers sensible group-level differences under DP (paid students trend slightly higher, both privately and non-privately), showing that group comparisons remain informative even with noise added to each group independently.
- Overall, PyDP's high-level bounded-aggregate API reproduces the same accuracy/privacy trade-off seen in the from-scratch Laplace mechanism of Task 1, but packaged as ready-to-use primitives (BoundedMean, Count, Min, Max, Median, BoundedStandardDeviation) with explicit lower/upper bounds instead of manually specified sensitivities.